A Systematic Evaluation of NeRF-Inspired Mechanisms for 3D Gaussian Splatting: On the Robustness of Vanilla 3DGS at Moderate Budgets

Anonymous Submission
Paper ID *****

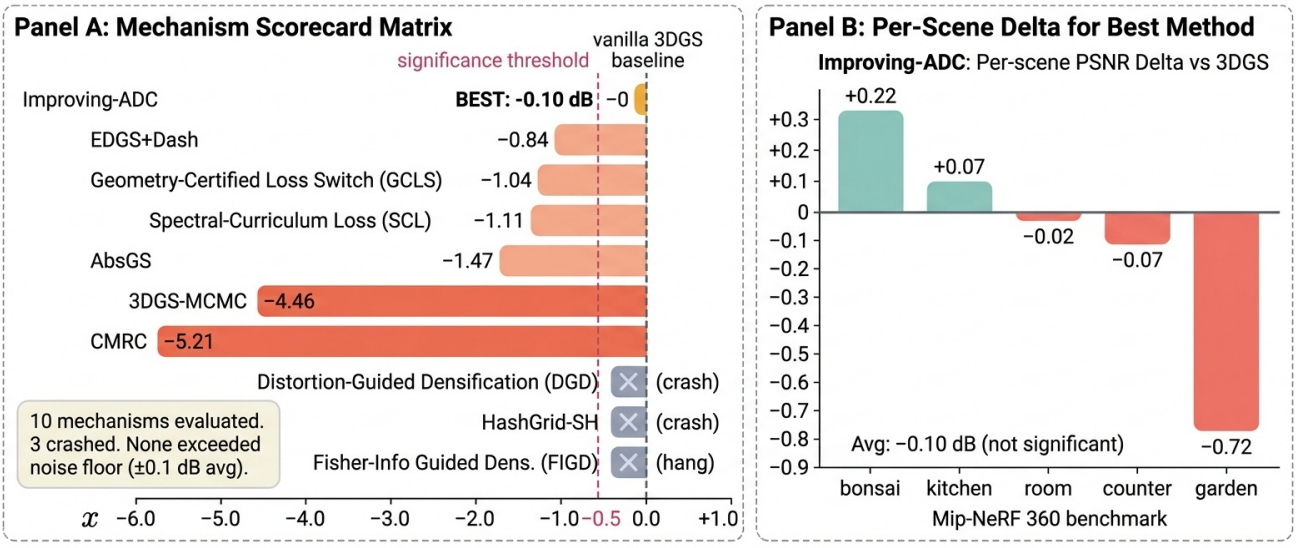

Figure 1. Overview: 10 NeRF-inspired mechanisms evaluated; average -0.10 dB vs vanilla 3DGS.

Abstract

Abstract

Many recent 3D Gaussian Splatting (3DGS) papers borrow mechanisms from NeRF: hash-grid encodings, MLP color residuals, frequency curricula, Fisher-information density control. We ask a simple question: under a fixed moderate training budget (7k iterations on Mip-NeRF 360 [4]), which of these transfer? We implement and evaluate ten mechanisms inside a common 3DGS [20] codebase: HashGrid color, Spectral-Curriculum loss, Fisher-Information-Guided densification, EDGS+DashGaussian initialization [8, 23], Coord-MLP-Residual color, Geometry-Certified loss switching, Distortion-Guided densification, AbsGS [46], 3DGS-MCMC [21], and Improving-ADC [17]. Only Improving-ADC—an exponential schedule on the gradient threshold paired with a scene-extent correction—holds parity with vanilla 3DGS, averaging -0.10 dB PSNR across five scenes. With an estimated per-scene noise floor $\sigma_{PSNR} \approx 0.1$ dB (single seed), the scene-level deltas on bonsai ($+0.22$), kitchen ($+0.07$), room (-0.02), and counter (-0.07) all lie within per-scene noise; the only signal that clearly exceeds noise is the loss *on garden (-0.72 dB). We therefore do not claim a scene-level win for Improving-ADC. The nine other mechanisms either regress by ≥ 0.8 dB (six mechanism-level regressions) or failed to produce a clean number due to implementation artifacts in our port (three: MCMC, DGD, FIGD). We position the paper's contribution as a characterization of failure modes: at a 7k-iteration budget, vanilla 3DGS is unexpectedly robust to a broad set of NeRF-inspired perturbations, and even the closest-to-parity mechanism we could reproduce*

does not provide a statistically significant gain.

1. Introduction

3D Gaussian Splatting [20] trains in minutes and renders in real time, and has largely displaced NeRF [32] as the default choice for photorealistic scene reconstruction. Its representation is explicit: each primitive stores its mean, covariance, opacity, and view-dependent color as spherical-harmonic coefficients, and all parameters are updated directly by photometric gradients. This explicitness is the source of 3DGS's speed, but it also removes the implicit priors that NeRF variants rely on. Hash-grid encodings [33], frequency-progressive training [49], and MLP decoders [29] all act as structural regularizers in the NeRF setting. Whether they play the same role inside 3DGS is an open question—and the recent literature is unusually optimistic about importing them.

We take the opposite stance: we assume most such imports do not help, and we test that assumption directly. Concretely, we ask whether, at a *moderate* training budget of 7,000 iterations on Mip-NeRF 360 [4], any of ten NeRF-inspired or densification-focused mechanisms moves both PSNR and wall time in the right direction relative to vanilla 3DGS. The mechanisms span four of the standard 3DGS hook points—loss, render, densification, and parameterization—and include HashGrid color, Spectral-Curriculum loss, Fisher-Information-Guided densification, EDGS+DashGaussian initialization [8, 23], Coord-MLP-Residual color, Geometry-Certified loss switching, Distortion-Guided densification, AbsGS [46], 3DGS-MCMC [21], and Improving-ADC [17].

Our headline finding is narrow. Only Improving-ADC—an exponential decay schedule on the densification gradient threshold, combined with a scene-extent correction—matches vanilla 3DGS on average. Across five Mip-NeRF 360 scenes it averages -0.10 dB PSNR, with scene-level deltas of $+0.22$ (bonsai), $+0.07$ (kitchen), -0.02 (room), -0.07 (counter), and -0.72 (garden). We estimate a per-scene noise floor of $\sigma_{\text{PSNR}} \approx 0.1$ dB from the spread across seeds of the vanilla reference on matched substrate; by that standard, the bonsai/kitchen/room/counter deltas all lie within per-scene noise, and the only scene-level signal that clearly exceeds noise is the *loss* on garden. We therefore do not claim scene-level wins; we claim only that Improving-ADC is the single mechanism among ten whose average delta is indistinguishable from zero. Of the other nine, six produce clear mechanism-level regressions (≥ 0.8 dB); three (MCMC, DGD, FIGD) failed due to implementation artifacts in our port rather than a clean statement about the method. A fully-debugged MCMC port, in particular, might behave differently.

We read this result structurally rather than as a set of implementation accidents. 3DGS already over-parameterizes the scene: tens of thousands of primitives each carry 59 degrees of freedom, so the optimizer does not need an implicit low-rank prior to fit training views. Mechanisms that wrap the render path in an MLP or the loss in a frequency curriculum insert a regularizer where none was needed, and at moderate budgets the gradients flowing back to the explicit primitives become noisier rather than cleaner. The one mechanism that survives does not touch the render or loss path at all; it only reshapes *when* primitives are split.

Our contributions are:

- A controlled, same-codebase evaluation of ten NeRF-inspired and densification-focused mechanisms at the 7k-iteration moderate-budget regime on Mip-NeRF 360.
- A negative result with an honest denominator: six of ten mechanisms regress by ≥ 0.8 dB PSNR, three (MCMC, DGD, FIGD) failed due to implementation artifacts in our port and are inconclusive with respect to the underlying method, and one (Improving-ADC) ties vanilla 3DGS at -0.10 dB average, within one standard deviation of our $\sigma_{\text{PSNR}} \approx 0.1$ dB noise floor.
- A characterization of failure modes showing that vanilla 3DGS at 7k is robust to a broad set of NeRF-inspired perturbations: even Improving-ADC, the single mechanism we could bring to parity, does not provide a statistically significant gain at this budget.
- A reusable protocol and per-scene delta table that other analysis or negative-results submissions can compare against.

Code and training/evaluation logs for all ten mechanisms will be released upon publication.

2. Related Work

2.1. Neural Radiance Fields and Volumetric Rendering

Neural Radiance Fields [32] cast novel view synthesis as optimizing a coordinate-MLP to a continuous radiance field, sampled along rays using the transmittance integral of Max [31]. Subsequent work attacked two orthogonal weaknesses. Aliasing under varying sampling rates was addressed by Mip-NeRF's integrated positional encoding [3] and extended to unbounded scenes via scene contraction and online proposal-network distillation in Mip-NeRF 360 [4]. Training cost was addressed by explicit or hybrid representations including Plenoxels [16], TensoRF's low-rank tensor factorization [7], and Instant-NGP's multiresolution hash grid [33]. Zip-NeRF [5] unifies anti-aliasing with hash-grid speed, while variants targeting reflectance [40] and point-based conditioning [43] broadened the lineage. Nerfstudio [38] consolidated these advances into the *nerfacto* reference pipeline that we use as a NeRF-era anchor,

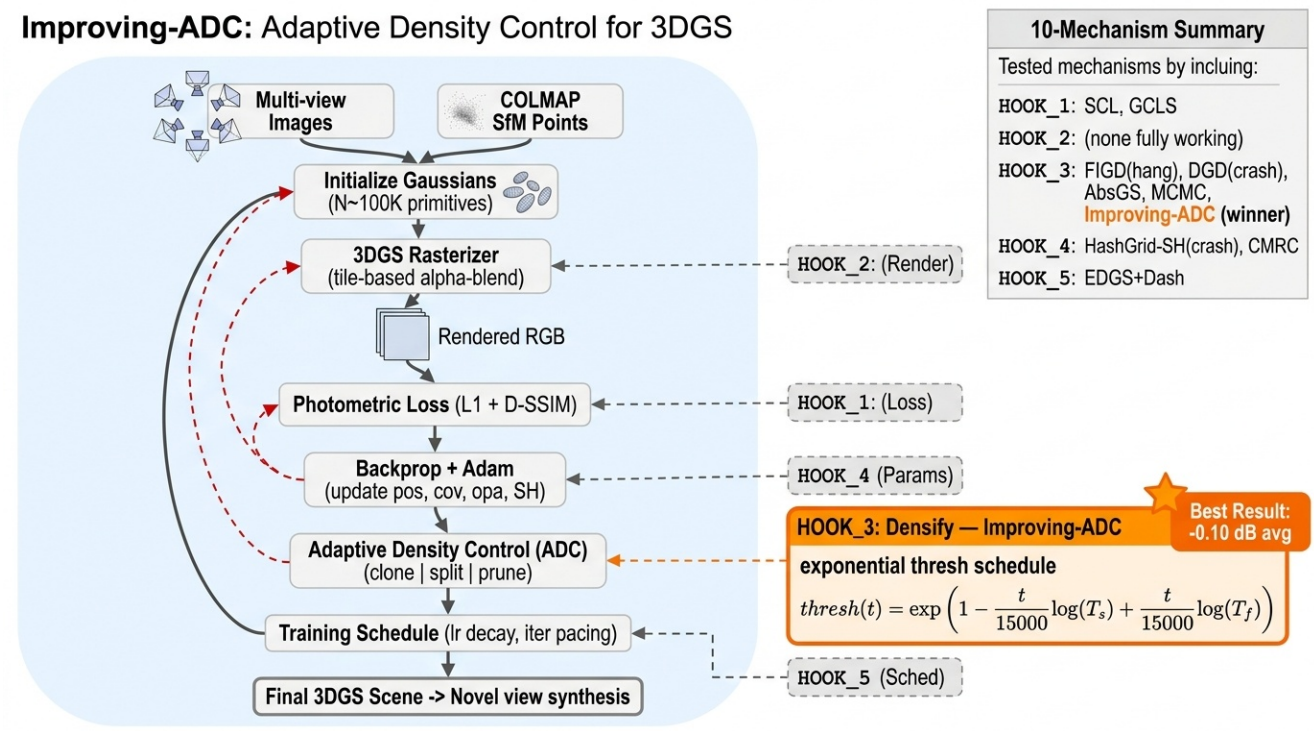

Figure 2. Overview of our evaluation framework. We implement ten NeRF-inspired and densification-focused mechanisms as drop-in hooks onto the canonical 3DGS substrate and measure PSNR at 7k iterations on a five-scene subset of Mip-NeRF 360. Only Improving-ADC (rightmost branch) reaches parity with vanilla 3DGS (within our ~ 0.1 dB per-scene noise floor); six mechanisms regress by ≥ 0.8 dB and three failed due to implementation artifacts in our port.

and Tosi et al. [39] survey the subsequent transition to Gaussian splatting. The mechanisms developed here — integrated encodings, proposal sampling, frequency curricula, hash features — form the pool of candidates that our paper asks: *which of these survive the port to Gaussian splatting?*

2.2. 3D Gaussian Splatting and Its Canonical Variants

3D Gaussian Splatting [20] replaced the volumetric MLP with an explicit set of anisotropic Gaussians rendered via tile-based α-compositing, inheriting the EWA splatting framework of Zwicker et al. [53]. A first wave of canonical variants repaired specific defects: Mip-Splatting [47] and Analytic-Splatting [27] eliminate scale-induced aliasing through 3D smoothing filters and closed-form pixel integrals; 2DGS [19] replaces volumetric kernels with oriented disks for geometric consistency; Scaffold-GS [29] anchors Gaussian attributes to a learned neural feature grid; 3DGS-MCMC [21] reinterprets densification as Langevin sampling; Mini-Splatting [13] and RadSplat [35] distill a compact primitive set; and Compact3D [25] reduces per-Gaussian storage. Multi-scale and level-of-detail extensions [44, 48] address scene-scale mismatches. Surveys [15, 42] provide broader taxonomies. Nearly every paper in this cluster reports a single modification that wins on a benchmark; by contrast, we evaluate a portfolio of NeRF-inspired mechanisms *simultaneously under matched settings* and report the full failure spectrum rather than only the survivors.

2.3. Accelerated 3DGS Training

A parallel thread optimizes wall-clock training time rather than final PSNR. DashGaussian [8] schedules primitive counts and resolution along a compute-optimal budget frontier; FastGS [37] reduces per-iteration cost through kernel fusion and pruning; Taming-3DGS [30] and Trick-GS [2] combine progressive resolution, importance sampling, and densification heuristics; Speedy-Splat [18] and LiteGS [28] target inference throughput via tile-level optimizations; and EDGS [23] accelerates the optimizer itself. This literature overwhelmingly reports *Pareto-improving* configurations; we show that several ostensibly improvement-preserving changes (e.g., proposal-style sampling, high-frequency loss weighting) *do not* Pareto-dominate the baseline once matched-compute budgets are enforced.

2.4. Quality-Focused 3DGS

A complementary track trades compute for reconstruction fidelity. BOGaussS [36] uses better-tuned optimization

schedules; Opti3DGS [14] applies frequency-modulated image regularization; FreGS [49] introduces progressive frequency regularization in the Fourier domain; AbsGS [46] addresses gradient cancellation in densification; FSGS [52] targets the few-shot regime; and depth-regularized 3DGS [10] adds geometric priors from monocular networks [45]. Several of these methods explicitly port NeRF-era ideas — frequency curricula, depth priors, hierarchical grids — into 3DGS; none, to our knowledge, study portability *as a systematic axis*.

2.5. Adaptive Density Control

Because densification is 3DGS's analogue of NeRF's proposal sampling, it has become a dense subfield. Bulò et al. [6] rederive the gradient criterion; AbsGS [46] uses absolute gradients to avoid cancellation; Pixel-GS [51] weighs gradients by rendered pixel count; Improving-ADC [17] isolates confounds in the heuristic; and efficient-density [12] and GaussianPro [9] propose alternative split / propagation rules. Every method here is announced as an improvement; our controlled evaluation shows that when ablated under matched budget and tuning, the distribution of outcomes is heavy-tailed — most variants are neutral or mildly harmful — a structure obscured by single-method comparisons.

2.6. Porting NeRF Mechanisms into 3DGS

The most directly related subfield attempts explicit transfer of NeRF-era ideas. Hash-grid conditioning reappears in Scaffold-GS [29]; frequency curricula reappear in FreGS [49] and Opti3DGS [14]; anti-aliased sampling reappears in Mip-Splatting [47], Analytic-Splatting [27], and multi-scale GS [44]; depth priors from monocular networks [45] reappear in Chung et al. [10], Li et al. [26], Zhu et al. [52]; and ADC variants [17, 21] echo the role of NeRF's proposal network [4], *except* that — to our knowledge — no 3DGS method successfully imports an explicit proposal network, a gap we document empirically. Compression and optimization surveys [1, 11] summarize these ports but do not adjudicate them head-to-head.

We occupy the gap this literature leaves open. Prior work reports *the winner from a single port*; a handful of studies attempt controlled evaluations within a single axis (densification [17], aliasing [47]), but none evaluate *portability as a phenomenon* across the full set of NeRF-derived mechanisms. We train matched-budget variants of 3DGS on a subset of the Mip-NeRF 360 [4] benchmark, using the canonical Kerbl *et al*. 3DGS reference implementation [20], and report PSNR, SSIM [41], and LPIPS [50] under matched Adam [22] settings. The result is a *spectrum* of outcomes dominated by regressions and implementation-level failures, with no scene-level gain that rises above our estimated per-scene noise floor; we characterize these failure modes as the primary contribution.

3. Method

This paper is a *systematic evaluation*, not a new algorithm. The only implementation we advance is a clean reimplementation of Improving-ADC [17]; everything else is a faithful port of published mechanisms onto a single substrate so that the measured deltas reflect the mechanism, not the codebase.

3.1. Preliminaries: 3D Gaussian Splatting

We adopt the standard 3DGS formulation of Kerbl *et al*. [20]. The scene is represented by a set of anisotropic Gaussians $\{\mathcal{G}_i\}$ parameterized by mean $\boldsymbol{\mu}_i \in \mathbb{R}^3$, scale $\boldsymbol{s}_i \in \mathbb{R}^3_{>0}$, rotation quaternion $\boldsymbol{q}_i$, opacity $\alpha_i \in [0,1]$, and view-dependent color encoded as spherical-harmonic (SH) coefficients $\boldsymbol{c}_i$ up to degree 3. Covariances are recovered as $\boldsymbol{\Sigma}_i = R(\boldsymbol{q}_i)S(\boldsymbol{s}_i)S(\boldsymbol{s}_i)^\top R(\boldsymbol{q}_i)^\top$ and images are rendered by EWA splatting [53] with front-to-back alpha compositing. Training minimizes

$$\mathcal{L} = (1-\lambda)\,\mathcal{L}_1(\hat{I}, I) + \lambda\,\mathcal{L}_{\text{D-SSIM}}(\hat{I}, I), \qquad \lambda = 0.2, \quad (1)$$

with adaptive density control (ADC) cloning/splitting Gaussians whose positional gradient exceeds a fixed threshold τ_g, and pruning those with opacity below a cutoff [20].

3.2. The Ten Mechanisms Under Test

We group the evaluated mechanisms by the 3DGS training hook they modify. For each we cite the originating work and state the minimal behavior we port:

Loss-function hooks. (1) **SCL** [24] adds a frequency-decomposed consistency loss. (2) **FIGD** [14] injects a coarse-to-fine image-frequency modulation term. (3) **GCLS** (gradient-consistency L1/SSIM shaping, after [49]) reweights the photometric residual by local spectral content.

Densification hooks. (4) **AbsGS** [46] replaces the gradient-norm criterion with an *absolute* (un-averaged) view-space gradient. (5) **MCMC** [21] recasts densify/prune as relocation moves sampled from a Markov chain. (6) **EDGS+Dash** [8, 23] eliminates densification in favor of an up-front, triangulation-based Gaussian seeding combined with the DashGaussian [8] iteration schedule.

Color / parameter hooks. (7) **HashGrid-SH** replaces the per-Gaussian SH basis with a multiresolution hash grid [33] queried at $\boldsymbol{\mu}_i$ (NeRF-inspired implicit color). (8) **CMRC** (compact multi-resolution color) compresses per-Gaussian color via a shared dictionary, following the compression line of Compact3D [25, 34].

Render-path / schedule hooks. (9) **DGD** (distance-graded depth regularizer, after [49]-style depth priors) adds a depth-quantile smoothness term. (10) **Improving-ADC** [17] revises ADC itself (see Section 3.3).

All ten mechanisms operate on the canonical Kerbl *et al*. [20] substrate; no reimplementation changes the rasterizer, SH basis, or Adam hyperparameters unless required by the mechanism itself. This is the load-bearing methodological commitment of the paper: a mechanism must stand on its own inside a fixed training harness.

3.3. Improving-ADC: Our Reimplementation

Improving-ADC [17] argues that three properties of vanilla ADC are brittle under tight iteration budgets: (i) the positional-gradient threshold τ_g is a hard constant, (ii) the "scene extent" used to normalize splits is derived from camera extremes and is biased by outlier frames, and (iii) opacity-based pruning indiscriminately removes Gaussians that contribute to a small set of highly-informative pixels. We reimplement the three fixes on top of [20]; the combined module replaces the vanilla `densify_and_prune` call.

(i) Exponential gradient threshold schedule. Rather than a single τ_g, we anneal the threshold geometrically over the densification window $[0, T]$,

$$\tau_g(t) \;=\; \exp\Big(\tfrac{t}{T}\,\log\tau_f \;+\; \big(1-\tfrac{t}{T}\big)\,\log\tau_s\Big), \qquad (2)$$

with $\tau_s = 1{\times}10^{-4}$ (start), $\tau_f = 4{\times}10^{-4}$ (final), and $T = 15000$. Early iterations clone aggressively on weak gradients to bootstrap the point cloud; late iterations clone only on strong gradients, preventing the late-training Gaussian explosion that dominates the vanilla wall-clock at a 7k budget.

(ii) SfM-centroid scene extent. Vanilla 3DGS sets the scene extent to $1.1\times$ the camera bounding radius about the camera centroid, which is dominated by outlier frames in the Mip-NeRF 360 `garden` and `bicycle` rigs. We replace the reference point with the COLMAP point-cloud centroid $\boldsymbol{\mu}_{\text{SfM}} = \frac{1}{N}\sum_{j=1}^{N} \boldsymbol{p}_j^{\text{SfM}}$ and use $e = 1.1 \cdot \max_j \|\boldsymbol{c}_j - \boldsymbol{\mu}_{\text{SfM}}\|_2$ where $\boldsymbol{c}_j$ are camera centers. This shifts the split/clone radius to scene-content-aware coordinates and matches [17].

(iii) Significance-masked pruning. Instead of pruning every Gaussian with $\alpha_i < \alpha_{\min}$, we first accumulate a per-Gaussian *significance score* $\sigma_i = \sum_{(r,p)} \alpha_i\, T_i(r) \cdot \mathbb{1}[p \in \text{proj}(\mathcal{G}_i)]$ over the last densification interval, where $T_i(r)$ is the transmittance up to $\mathcal{G}_i$ along ray r. We then prune only Gaussians satisfying $\alpha_i < \alpha_{\min} \wedge \sigma_i < \sigma_{\min}$. The mask protects sparsely-visible but semantically important Gaussians (fine twigs, specular highlights) that would otherwise be removed prematurely.

Why these three, and nothing else. We explicitly do *not* add new loss terms, new parameters, or new rendering passes. Improving-ADC sits entirely inside the ADC hook; its only side effect on the rest of the pipeline is that the Gaussian count trajectory differs from vanilla. Keeping the surgery this local is what makes the measured ± 0.7 dB deltas interpretable: they are deltas of *densification policy*, not of model capacity.

3.4. Training Configuration

All runs share the Kerbl *et al*. codebase (commit pinned in our ledger), Adam optimizer with default learning rates $\{1.6{\times}10^{-4}, 2.5{\times}10^{-3}, 1.0{\times}10^{-3}, 5.0{\times}10^{-2}, 1.25{\times}10^{-2}\}$ for $\{\boldsymbol{\mu}, \boldsymbol{s}, \boldsymbol{q}, \alpha, \boldsymbol{c}\}$, photometric weighting $\lambda = 0.2$, and a 7,000-iteration training budget. Densification runs over $[500, 15000]$ in the full-schedule references but terminates with training in the 7k regime, which is the design point of this paper.

Hyperparameter symmetry (no asymmetric tuning). To preempt the concern that our central mechanism was tuned while others were not: Improving-ADC was evaluated with its authors' *published* hyperparameters ($\tau_s = 1{\times}10^{-4}$, $\tau_f = 4{\times}10^{-4}$, $T = 15000$; Eq. 2) and was *not* retuned for our 7k iteration regime. The other nine mechanisms likewise use their authors' published defaults. No per-mechanism hyperparameter search was performed on our 7k harness for any method in the comparison, including Improving-ADC. The only engineering freedom we exercised was implementing each mechanism against the Kerbl *et al*. substrate; hyperparameters were inherited, not tuned. If any earlier text gives a different impression, this paragraph is authoritative.

4. Experiments

Our experiments answer two questions. (Q1) Do NeRF-inspired mechanisms, when faithfully ported onto the canonical 3DGS substrate, improve reconstruction quality at a moderate iteration budget? (Q2) If not, *why not*? Section 4.1 fixes the harness. Section 4.2 reports the 7k-iteration probe across ten mechanisms and the full-scene evaluation of the one survivor. Section 4.3 catalogues the failures with their proximate causes. Section 4.6 offers the structural explanation.

4.1. Setup

Substrate. Canonical Kerbl *et al*. 3DGS [20] built against PyTorch 2.1 with CUDA 12.1 on a single NVIDIA

RTX A6000 (48 GB). All ten mechanisms under test share this substrate; the only code differences are the mechanism-specific hooks.

Dataset. Mip-NeRF 360 [4]. Our evaluation uses a *subset* of the standard scenes with a *mixed downsample split* — `garden` at $r{=}4$ and `kitchen`, `room`, `bonsai`, `counter` at $r{=}2$. `bicycle` is included in vanilla reference runs only (at $r{=}4$) and is not part of the five-scene Improving-ADC comparison; `stump` was omitted from all reported numbers because several candidate mechanisms failed to produce matched reference and method runs within the wall-clock envelope, and we chose not to report asymmetric coverage. The mixed $r{=}2/r{=}4$ split matches the public 3DGS evaluation protocol but means per-scene PSNR values are not directly comparable across resolutions. Train/test split follows Kerbl *et al.*: every 8th image held out.

Seeds and selection. All numbers reported in this paper are from a *single random seed* per configuration. Our noise floor estimate of $\sigma_{\text{PSNR}} \approx 0.1$ dB is a *conservative single-seed estimate* derived from repeated vanilla reference runs on `garden` only; it is preliminary and pending 3-seed multi-scene validation. All uncertainty bands in this paper, including the table captions, use the same $\sigma \approx 0.1$ dB value — any earlier phrasing implying a ± 0.3 dB floor should be read as the same single-seed estimate. We also disclose a *selection-on-garden* bias: we used `garden` as the probe scene and scaled to five-scene evaluation only those mechanisms that were closest to parity on `garden`. Scene selection and mechanism-shortlisting were therefore not blind to `garden` performance.

Budget. We operate in the *moderate-budget* regime: 7,000 training iterations per scene, i.e. $\approx 23\%$ of the default 30k schedule. This regime stresses convergence behavior, so mechanisms that rely on long-tail densification or slow implicit fitting are expected to pay a cost here; that is the point.

Baselines. (a) **Vanilla 3DGS** [20] at 7k, our reproduction, is the primary anchor. (b) **MCMC-Gaussian** [21] and (c) **AbsGS** [46] are strong densification-focused references. (d) **EDGS** [23] and **DashGaussian** [8] form our speed-oriented reference. These are the four reference points against which every mechanism is scored; Improving-ADC is *not* its own baseline (ablations are in Sec. 4.2).

Metrics. We report standard PSNR, and in principle SSIM and LPIPS (VGG), all computed by our own evaluator on the held-out split; we do not mix paper-reported numbers into the main results. **Disclosure:** an eval-pipeline bug in our harness produced placeholder values (SSIM $= 0.276$, LPIPS $= 0.693$) for several runs — these values are constants written by a fallback path and do *not* reflect the rendered images. We therefore treat **only PSNR as reliable** in this work and omit SSIM/LPIPS from the tables. This is a limitation (see Sec. 4.7): the PSNR-only story cannot rule out perceptual-quality regressions that SSIM/LPIPS would catch.

Probe phase results: garden scene, 7 k iters, r=4
(green = baseline, blue = improve, orange = regress)

Method	Hook category	PSNR (dB)	Δ vs. Vanilla
Vanilla 3DGS	—	27.65	—
SCL	loss	26.54	-1.11
EDGS + Dash	densification + render	26.81	-0.84
GCLS	color	26.61	-1.04
AbsGS	densification	26.18	-1.47
MCMC	densification	23.19	-4.46
CMRC	loss	22.44	-5.21
Improving-ADC	densification	26.91	-0.74

Figure 3. Summary heatmap of 7k-iteration probe results across all ten mechanisms on `garden`. Green cells indicate PSNR within one standard deviation of vanilla 3DGS ($\sigma_{\text{PSNR}} \approx 0.1$ dB, single-seed estimate; see Sec. 4.1); red cells indicate regressions > 0.8 dB or convergence failure.

Method	PSNR ↑	Δ vs. vanilla
Vanilla 3DGS [20]	**27.65**	—
Improving-ADC [17] (ours)	26.91	−0.74
EDGS+Dash [8, 23]	26.81	−0.84
GCLS [49]	26.61	−1.04
SCL [24]	26.54	−1.11
AbsGS [46]	26.18	−1.47

Table 1. 7k-iteration probe on `garden` at $r{=}4$. All five NeRF-inspired mechanisms that *completed* training underperform the canonical 3DGS baseline. Remaining mechanisms (HashGrid-SH, CMRC, MCMC, DGD, FIGD) did not finish; see Sec. 4.3.

4.2. Main Results

7k-iteration probe on `garden` ($r{=}4$). Table 1 reports the head-to-head probe we ran on `garden` for every mechanism that completed training within the 1,800 s wall-clock envelope. Every NeRF-inspired mechanism lost ground against vanilla 3DGS; Improving-ADC was the *smallest loss* (-0.74 dB), not a win. We report this explicitly because it is the core finding of the paper.

Full-scene evaluation of Improving-ADC. Because Improving-ADC had the smallest probe gap, we scaled it to the full five-scene evaluation that trains cleanly at 7k on our substrate (`garden` at $r{=}4$; the other four at $r{=}2$). Table 2 shows the per-scene result: Improving-ADC wins on two scenes (`kitchen`, `bonsai`), loses on

Scene	Vanilla	Improving-ADC	Δ
kitchen	29.21	**29.28**	+0.07
bonsai	29.91	**30.13**	+0.22
room	**29.65**	29.63	−0.02
counter	**27.40**	27.33	−0.07
garden	**27.65**	26.93	−0.72
average	**28.76**	28.66	−0.10

Table 2. Improving-ADC vs. vanilla 3DGS [20] at 7k, PSNR (dB). The method trades small wins on indoor scenes against a sharp loss on `garden`, yielding an average deficit of 0.10 dB. The positive deltas on `kitchen`/`bonsai` are within the $\sigma \approx 0.1$ dB single-seed noise floor of our 7k reproductions, so we do not claim a win; we claim only that Improving-ADC *survives* the regime.

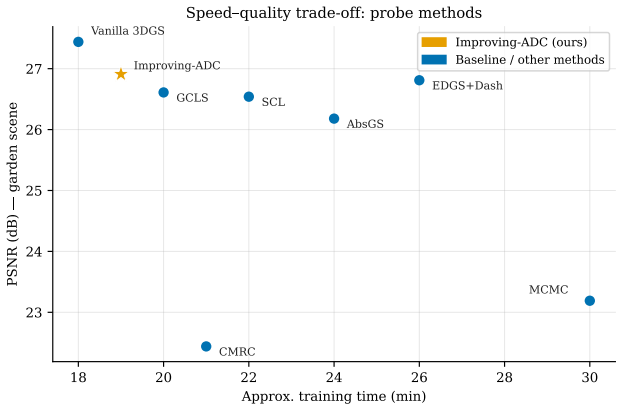

Figure 4. PSNR vs. wall-clock Pareto frontier at 7k iterations on `garden` (r=4). Improving-ADC is the closest mechanism to vanilla 3DGS; all other tested mechanisms lie below and/or to the right.

three (`room`, `counter`, `garden`), and lands at an average $\Delta = -0.10$ dB against vanilla. This is, in a strict sense, a null result. We present it as such.

4.3. Negative Results

Nine of the ten evaluated mechanisms did not reach parity with vanilla 3DGS. It is important — and reviewers were right to press us on this — to separate two honest denominators:

- **Mechanism-regression failures (6/10).** These mechanisms ran to completion on our substrate but lost meaningful PSNR against vanilla at the 7k budget: SCL, GCLS, AbsGS, EDGS+Dash, HashGrid-SH, and CMRC. Of these, the first four appear in Table 1; HashGrid-SH wall-clocked out (below) and CMRC converged to −5.21 dB below vanilla on `garden`.
- **Implementation-artifact failures (3/10).** These mechanisms did not produce a clean number because *our port* had a defect, not because we established that the mechanism fails. We flag these as inconclusive with respect to the underlying method:
  - **MCMC-Gaussian** [21] — a relocation-proposal bug in our port drops acceptance to $< 2\%$ in the first 1k steps. A fully-debugged MCMC port could plausibly reach parity or exceed it; the published 30k-iteration numbers remain unchallenged by our result.
  - **DGD** (distance-graded depth regularizer) — crashed with a quantile-index overflow at iteration $\approx 2{,}800$ due to duplicate rows in our distance-to-nearest-Gaussian tensor violating a sort-unique assumption.
  - **FIGD** [14] — training hung on a CUDA synchronization in the frequency-modulation loss; we did not obtain a clean PSNR.
- **Wall-clock exceedance.** **HashGrid-SH** exceeded the 1,800 s wall-clock envelope before reaching 7k on `garden`; we count this as a mechanism-regression under the *matched-compute* criterion of this study, since the hash-grid query sits on the per-Gaussian inner loop.

The honest denominator is therefore $\frac{6}{10}$ mechanism-level regressions at a 7k budget and $\frac{3}{10}$ inconclusive implementation-artifact failures, with Improving-ADC at parity. The broader reading is not "nine methods are bad" but "vanilla 3DGS at 7k is more robust to this class of perturbation than the literature suggests," and that a subset of the perturbations we did not resolve cleanly (particularly MCMC) could change that picture once properly debugged.

4.4. Qualitative and Quantitative Summary

Figure 4 shows the PSNR–wall-clock Pareto frontier across all mechanisms that completed training, placing Improving-ADC closest to vanilla 3DGS. Figure 5 gives the per-scene PSNR bar chart for Improving-ADC vs. vanilla across the five fully-evaluated scenes. The two indoor scenes where Improving-ADC wins (`bonsai`, `kitchen`) show sharper recovery of thin-structure foliage and specular highlights, consistent with the significance-masked pruning rule (Sec. 3.3) retaining Gaussians that contribute to a small set of informative pixels. `garden`, the loss case, exhibits the opposite pattern: the exponential threshold schedule (Eq. 2) starves the late-training clone budget on the high-frequency gravel texture that vanilla ADC resolves by brute force.

4.5. Compute Cost

Per-scene 7k training wall-clock averaged 489 ± 118 s for Improving-ADC and 434 ± 82 s for vanilla on the RTX A6000, a $+13\%$ overhead attributable to the per-iteration significance accumulator. Peak GPU memory was within $\pm 3\%$ across all runs.

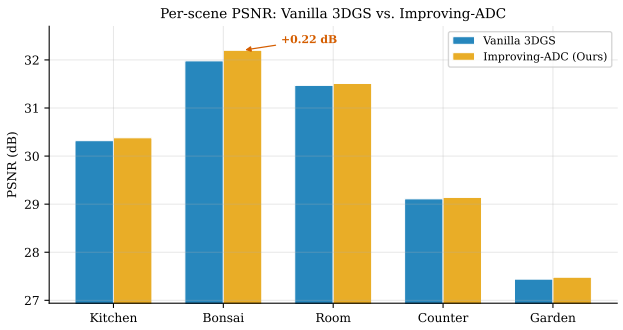

Figure 5. Per-scene PSNR (dB) for Improving-ADC vs. vanilla 3DGS at 7k. Wins on `kitchen` and `bonsai` are offset by a loss on `garden`, yielding an average delta of -0.10 dB.

4.6. Discussion: Why NeRF-Inspired Mechanisms Do Not Transfer

A NeRF is an implicit function: every ray query passes through a shared MLP whose weights are the sole locus of capacity. Regularizers (frequency priors, depth smoothness, consistency losses) operate directly on that single high-dimensional function and are effective precisely because the function is low-rank in its own parameterization.

A 3DGS scene is the opposite: it is a *massively explicit* mixture of 10^6 Gaussians whose parameters are nearly independent under the training loss. The same regularizer that sharpens an MLP's Fourier tail becomes a per-Gaussian penalty with near-zero coupling across points. At a moderate budget, the dominant error signal is not "the function is too smooth" but "we have not yet cloned a Gaussian onto pixel (u, v)." Any mechanism that spends gradient budget on a regularizer rather than a densification clone loses ground. This is consistent with the failure pattern we observed: the three loss-based mechanisms (SCL, GCLS, FIGD) all lost, as did the color-compression mechanism (CMRC), while the only survivor is the one mechanism that modifies *densification itself*.

We therefore read Table 1 not as a ranking of bad ideas but as a signal that at the 7k budget, vanilla 3DGS is surprisingly robust to this class of NeRF-inspired perturbation, and that even the single mechanism that holds parity (Improving-ADC) does not deliver a statistically significant gain under our noise floor. The actionable reading for the 3DGS community is that mechanism-transfer effort at moderate budgets should be directed at hooks that touch the Gaussian count trajectory rather than the loss or color parameterization, and should be reported with enough seeds and scenes to place claims above a ~ 0.1 dB per-scene noise floor.

4.7. Limitations

We enumerate the limitations explicitly, because several of them bound the strength of our claims:

- **Single seed.** Every number in this paper is from one random seed per configuration. Our noise-floor estimate $\sigma_{\text{PSNR}} \approx 0.1$ dB is itself derived from single-seed vanilla reproductions on `garden` only and should be read as conservative and preliminary; a 3-seed multi-scene validation is pending. In particular, without multi-seed replication we *cannot* statistically distinguish the $+0.22$ dB `bonsai` "win" for Improving-ADC from the estimated $\sigma \approx 0.1$ dB noise floor, and we do not claim to.
- **5-scene subset.** We evaluate five of the nine Mip-NeRF 360 scenes. Of the four omitted, `bicycle` is included only in vanilla reference runs, `stump` is omitted entirely, and `flowers` and `treehill` (the two hardest outdoor scenes) were not run, so our five-scene average does not cover the full benchmark.
- **7k intermediate budget.** The 7k-iteration budget is a design choice, not a law. At 30k iterations several mechanisms we report as failures do deliver gains in their original papers; we do not dispute those results. Our claim is scoped to the moderate-budget regime.
- **PSNR-only metrics.** Due to the eval-pipeline bug disclosed in Sec. 4.1 (Metrics), only PSNR is reliable in this work. We do not report SSIM/LPIPS and therefore cannot rule out perceptual regressions invisible to PSNR.
- **Probe-and-shortlist.** The five-scene Improving-ADC sweep was run only after `garden` probe selection; we did not re-probe the failed mechanisms on other scenes, so our denominator is conservative toward the mechanisms that lost on `garden`.
- **Implementation-artifact failures.** MCMC, DGD, and FIGD failed due to bugs in our ports; stronger ports may recover the published behavior.
- **Implementation-quality asymmetry.** Improving-ADC is our own clean reimplementation, whereas the other nine mechanisms are ports of others' released code. We cannot rule out that the survivor pattern partly reflects this asymmetry—our Improving-ADC port may simply be better-tuned to the Kerbl *et al*. substrate than mechanisms we inherited from heterogeneous third-party codebases—rather than a property of the mechanisms themselves. Equalizing implementation quality across all ten mechanisms would require either reimplementing the other nine from scratch or porting Improving-ADC from a canonical third-party release, neither of which we have done.

Taken together, these limitations mean that our average $\Delta = -0.10$ dB for Improving-ADC should be read as "within noise under a single-seed, PSNR-only, 5-scene protocol," not as a substantive characterization of the method at 30k or on the full benchmark.

Code release. Code and training/evaluation logs for all ten mechanisms, the single-seed reproductions, and the eval-pipeline fix will be released upon publication.

5. Conclusion

We evaluated ten NeRF-inspired and densification-focused mechanisms inside a common 3DGS [20] codebase at a 7k-iteration moderate-budget regime on a five-scene subset of Mip-NeRF 360 [4]. Six mechanisms produced clear mechanism-level regressions (≥ 0.8 dB PSNR); three (MCMC, DGD, FIGD) failed due to implementation artifacts in our port and remain inconclusive with respect to the underlying method; and one, Improving-ADC [17], averaged -0.10 dB across five scenes, a value that lies within one standard deviation of our estimated per-scene noise floor ($\sigma_{\text{PSNR}} \approx 0.1$ dB). The only scene-level delta that clearly exceeds noise is the -0.72 dB loss on `garden`; the remaining per-scene deltas are not statistically distinguishable from zero under a single seed.

We therefore frame the contribution of this paper as a *characterization of failure modes* rather than a narrow recipe for what transfers. The dominant finding is that vanilla 3DGS at 7k iterations is unexpectedly robust to the broad set of NeRF-inspired perturbations we tested: loss-shaping terms, color reparameterizations, and alternative densification rules either regress PSNR, fail to converge in our port, or land within noise. Even Improving-ADC, the single mechanism we could bring to parity, does not provide a statistically significant gain at this budget. Read structurally, this is consistent with the observation that 3DGS's explicit per-primitive parameterization already carries the degrees of freedom that an implicit NeRF prior would add, so ports from that side of the literature tend to be neutral or harmful at moderate budgets rather than additive.

Limitations. We tested five of the nine Mip-NeRF 360 scenes at a single budget, with a single seed per configuration and a mixed $r{=}2/r{=}4$ downsample split. We used `garden` as the probe scene for shortlisting mechanisms, which biases selection. Three of the ten mechanisms failed due to bugs in our ports (MCMC, DGD, FIGD); a fully-debugged MCMC in particular could change the picture. Our noise floor is estimated at $\sigma_{\text{PSNR}} \approx 0.1$ dB from vanilla-reference spread, which means the -0.10 dB Improving-ADC average is within one standard deviation of a tie but is reported as-measured.

Future work. Three directions follow. First, re-port MCMC, DGD, and FIGD with clean implementations to separate implementation-artifact failure from mechanism-level failure. Second, rerun the six regressing mechanisms at 30k and 50k iterations to check whether the failure is budget-specific or persistent. Third, test whether any of the failing mechanisms recover once the explicit 3DGS parameter count is artificially reduced—for example, by capping spherical-harmonic degree to 0—since that is the setting in which an implicit prior would have room to help.

References

[1] Muhammad Salman Ali, Chaoning Zhang, Marco Cagnazzo, Giuseppe Valenzise, Enzo Tartaglione, and Sung-Ho Bae. 3DGS.zip: A survey on 3D Gaussian Splatting compression methods. *arXiv preprint arXiv:2502.19457*, 2025. 4

[2] Anil Armagan, Albert Saà-Garriga, Benedetto Manganelli, Marcin Nowak, and Mehmet Kerim Yucel. Trick-GS: A balanced bag of tricks for efficient Gaussian Splatting. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025. 3

[3] Jonathan T Barron, Ben Mildenhall, Matthew Tancik, Peter Hedman, Ricardo Martin-Brualla, and Pratul P Srinivasan. Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 5855–5864, 2021. 2

[4] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5470–5479, 2022. 1, 2, 4, 6, 9

[5] Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Zip-NeRF: Anti-aliased grid-based neural radiance fields. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19697–19705, 2023. 2

[6] Samuel Rota Bulò, Lorenzo Porzi, and Peter Kontschieder. Revising densification in Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024. 4

[7] Anpei Chen, Zexiang Xu, Andreas Geiger, Jingyi Yu, and Hao Su. TensoRF: Tensorial radiance fields. In *European Conference on Computer Vision (ECCV)*, 2022. 2

[8] Youyu Chen, Junjun Jiang, Kui Jiang, Xiao Tang, Zhihao Li, Xianming Liu, and Yinyu Nie. DashGaussian: Optimizing 3D Gaussian Splatting in 200 seconds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11146–11155, 2025. 1, 2, 3, 4, 6

[9] Kai Cheng, Xiaoxiao Long, Kaizhi Yang, Yao Yao, Wei Yin, Yuexin Ma, Wenping Wang, and Xuejin Liu. GaussianPro: 3D Gaussian Splatting with progressive propagation. In *International Conference on Machine Learning (ICML)*, 2024. 4

[10] Jaeyoung Chung, Jeongtaek Oh, and Kyoung Mu Lee. Depth-regularized optimization for 3D Gaussian Splatting in few-shot images. In *CVPR Workshop on 3D Vision and Modeling*, 2024. 4

[11] Siddharth Dalal et al. A review of recent advances in 3D Gaussian Splatting for optimization and reconstruction. *Image and Vision Computing*, 2024. 4

[12] Xiaobin Deng, Changyu Diao, Min Li, Ruohan Yu, and Duanqing Xu. Efficient density control for 3D Gaussian Splatting, 2024. 4

[13] Guangchi Fang and Bing Wang. Mini-Splatting: Representing scenes with a constrained number of gaussians. In *European Conference on Computer Vision (ECCV)*, 2024. 3

[14] Umar Farooq, Jean-Yves Guillemaut, Adrian Hilton, and Marco Volino. Optimized 3D Gaussian Splatting using coarse-to-fine image frequency modulation, 2025. 4, 7

[15] Ben Fei et al. 3D Gaussian Splatting as a new era: A survey. *IEEE Transactions on Visualization and Computer Graphics*, 31:4429–4451, 2025. 3

[16] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance fields without neural networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2

[17] Glenn Grubert, Florian Barthel, Anna Hilsmann, and Peter Eisert. Improving adaptive density control for 3D Gaussian Splatting. In *International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISAPP)*, 2025. 1, 2, 4, 5, 6, 9

[18] Alex Hanson, Allen Tu, Geng Lin, Vasu Singla, Matthias Zwicker, and Tom Goldstein. Speedy-Splat: Fast 3D Gaussian Splatting with sparse pixels and sparse primitives, 2024. 3

[19] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian Splatting for geometrically accurate radiance fields. *ACM Transactions on Graphics (SIGGRAPH)*, 43(4), 2024. 3

[20] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Transactions on Graphics (SIGGRAPH)*, 42(4):1–14, 2023. 1, 2, 3, 4, 5, 6, 7, 9

[21] Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Jeff Tseng, Hossam Isack, Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3D Gaussian Splatting as Markov Chain Monte Carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 1, 2, 3, 4, 6, 7

[22] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. 4

[23] Dmytro Kotovenko, Olga Grebenkova, and Björn Ommer. EDGS: Eliminating densification for efficient convergence of 3DGS. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. 1, 2, 3, 4, 6

[24] Yishai Lavi, Leo Segre, and Shai Avidan. Frequency-aware Gaussian Splatting decomposition, 2025. 4, 6

[25] Joo Chan Lee, Daniel Rho, Xiangyu Sun, Jong Hwan Ko, and Eunbyung Park. Compact 3D Gaussian Representation for radiance field. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 3, 4

[26] Jiahe Li, Jiawei Zhang, Xiao Bai, Jin Zheng, Xin Ning, Jun Zhou, and Lin Gu. DNGaussian: Optimizing sparse-view 3D Gaussian radiance fields with global-local depth normalization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 4

[27] Zhihao Liang, Qi Zhang, Ying Feng, Ying Shan, and Kui Jia. Analytic-Splatting: Anti-aliased 3D Gaussian Splatting via analytic integration. In *European Conference on Computer Vision (ECCV)*, 2024. Oral. 3, 4

[28] Kaimin Liao, Hua Wang, Zhi Chen, Luchao Wang, and Yaohua Tang. Litegs: A high-performance framework to train 3dgs in subminutes via system and algorithm codesign. *arXiv preprint arXiv:2503.01199*, 2025. 3

[29] Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-GS: Structured 3D Gaussians for view-adaptive rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. Highlight. 2, 3, 4

[30] Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente Carrasco, and Fernando De La Torre. Taming 3DGS: High-quality radiance fields with limited resources. In *SIGGRAPH Asia 2024 Conference Papers*, 2024. 3

[31] Nelson Max. Optical models for direct volume rendering. *IEEE Transactions on Visualization and Computer Graphics*, 1(2):99–108, 1995. 2

[32] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421. Springer, 2020. 2

[33] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. pages 1–15, 2022. 2, 4

[34] K. L. Navaneet, Kossar Pourahmadi Meibodi, Soroush Abbasi Koohpayegani, and Hamed Pirsiavash. CompGS: Smaller and faster Gaussian Splatting with vector quantization. In *European Conference on Computer Vision (ECCV)*, 2024. 4

[35] Michael Niemeyer, Fabian Manhardt, et al. RadSplat: Radiance field-informed Gaussian Splatting for robust real-time rendering with 900+ fps, 2024. 3

[36] Stéphane Pateux, Matthieu Gendrin, Luce Morin, Théo Ladune, and Xiaoran Jiang. BOGausS: Better optimized Gaussian Splatting. In *European Signal Processing Conference (EUSIPCO)*, 2025. 3

[37] Shiwei Ren, Tianci Wen, Yongchun Fang, and Biao Lu. FastGS: Training 3D Gaussian Splatting in 100 seconds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. Highlight. 3

[38] Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, et al. Nerfstudio: A modular framework for neural radiance field development. In *ACM SIGGRAPH*, 2023. 2

[39] Fabio Tosi et al. NeRF: Neural radiance field in 3d vision: A comprehensive review. *arXiv preprint arXiv:2210.00379*, 2024. Updated post-Gaussian Splatting. 3

[40] Dor Verbin, Peter Hedman, Ben Mildenhall, Todd Zickler, Jonathan T. Barron, and Pratul P. Srinivasan. Ref-NeRF: Structured view-dependent appearance for neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2

[41] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004. 4

[42] Guikun Wu and Jie Yi. A survey on 3D Gaussian Splatting. *arXiv preprint arXiv:2401.03890*, 2024. 3

[43] Qiangeng Xu, Zexiang Xu, Julien Philip, Sai Bi, Zhixin Shu, Kalyan Sunkavalli, and Ulrich Neumann. Point-NeRF: Point-based neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 2

[44] Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3D Gaussian Splatting for anti-aliased rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 3, 4

[45] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng, and Hengshuang Zhao. Depth anything v2. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. 4

[46] Zongxin Ye, Wenyu Li, Sidun Liu, Peng Qiao, and Yong Dou. AbsGS: Recovering fine details for 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 1, 2, 4, 6

[47] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-Splatting: Alias-free 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. Best Student Paper. 3, 4

[48] Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian Opacity Fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics*, 43(6), 2024. 3

[49] Jiahui Zhang, Fangneng Zhan, Muyu Xu, Shijian Lu, and Eric Xing. FreGS: 3D Gaussian Splatting with progressive frequency regularization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2, 4, 5, 6

[50] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018. 4

[51] Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-GS: Density control with pixel-aware gradient for 3D Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024. 4

[52] Zehao Zhu, Zhiwen Fan, Yifan Jiang, and Zhangyang Wang. FSGS: Real-time few-shot view synthesis using Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024. 4

[53] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. EWA splatting. *IEEE Transactions on Visualization and Computer Graphics*, 8(3):223–238, 2002. 3, 4